Homework 2

Symbolic Music Generation

TA: 陳柏睿 (Bo-Rui Chen)
Office hour: Mon. 14:20~15:20 @BL505

Outline

- Overview
- Detailed Explanations
  - Basic Task: Unconditional Generation & Prompt Continuation
  - Bonus Task: Accompaniment Generation
- Submission
- Scoring
- Rules
- Timeline

Overview

- Homework objectives:
  - Learn to manipulate MIDI files and represent symbolic music as tokens
  - Learn to train an autoregressive model **from scratch** for symbolic music generation

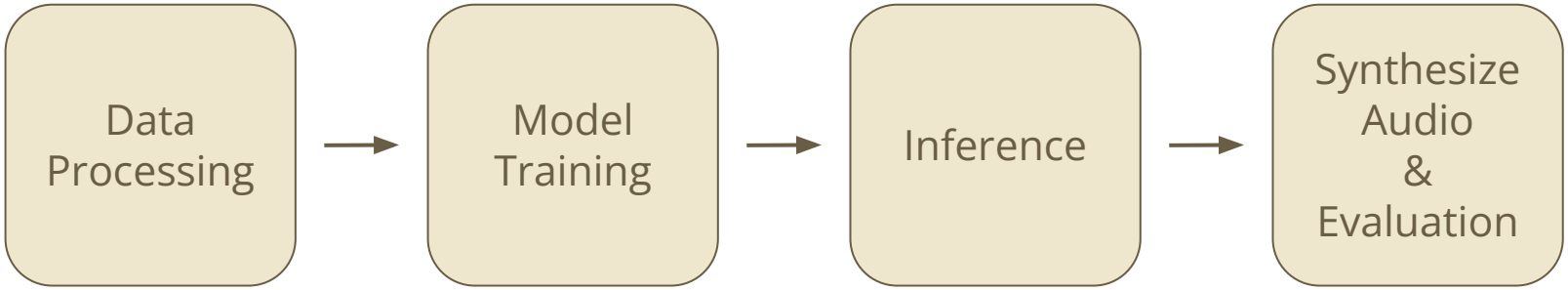
Overview

- General SOP for training/testing a symbolic generation model

  - Step 1: Prepare data
  - Step 2: Token representation
  - Step 3: Pick a model
  - Step 4: Train the model until the loss is sufficiently low
  - Step 5: Perform inference
  - Step 6: Listen to the generated pieces
  - Step 7: Run evaluation

Overview

- General SOP for training/testing a symbolic generation model

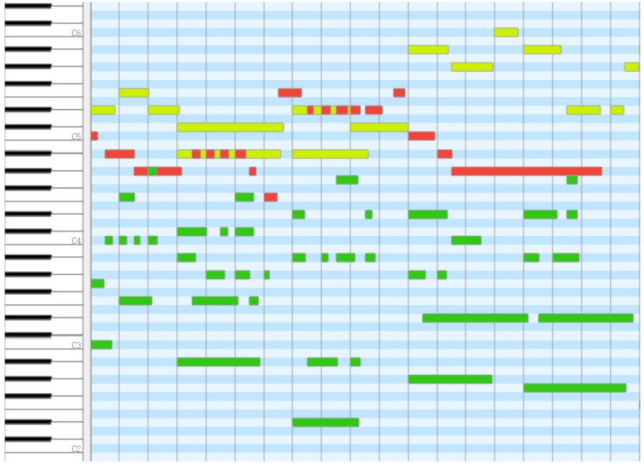


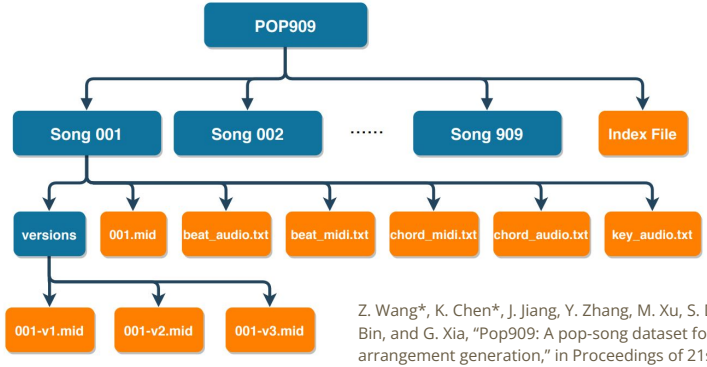
Detailed Explanations

- Dataset
- Token Representation
- Basic Tasks
  - a. Unconditional Generation
  - b. Prompt Continuation
- Bonus Task
  - Accompaniment Generation

Dataset

- In this homework, we use the POP909 dataset
  - 909 Mandopop piano performances, with labeled beats & chords
  - 3 tracks: MELODY, BRIDGE & PIANO
    - For the basic tasks, we merge all the three tracks together
    - For the bonus task, we only use the separated MELODY and PIANO tracks

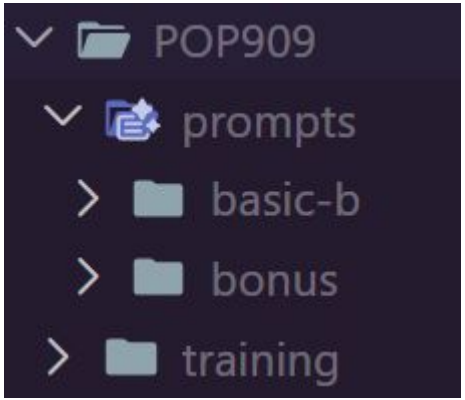




Z. Wang*, K. Chen*, J. Jiang, Y. Zhang, M. Xu, S. Dai, G. Bin, and G. Xia, "Pop909: A pop-song dataset for music arrangement generation," in Proceedings of 21st International Conference on Music Information Retrieval, ISMIR, 2020.

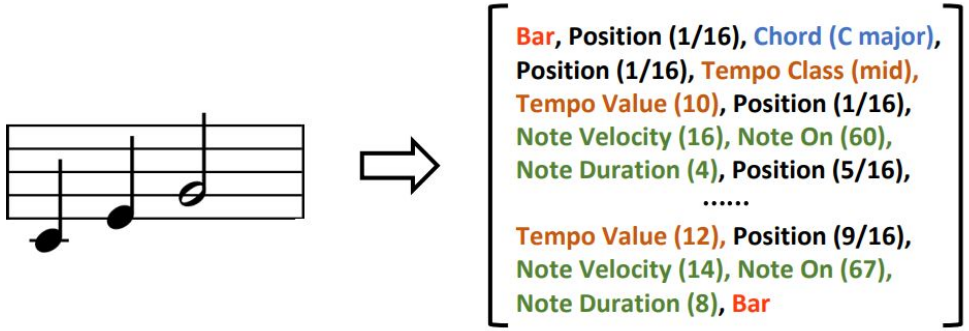
Dataset

- For those 909 songs, **896** will be used to as training set (under the *training/* folder) , and the remaining will be prompts for some tasks in this homework (under the *prompts/* folder)



Token Representation

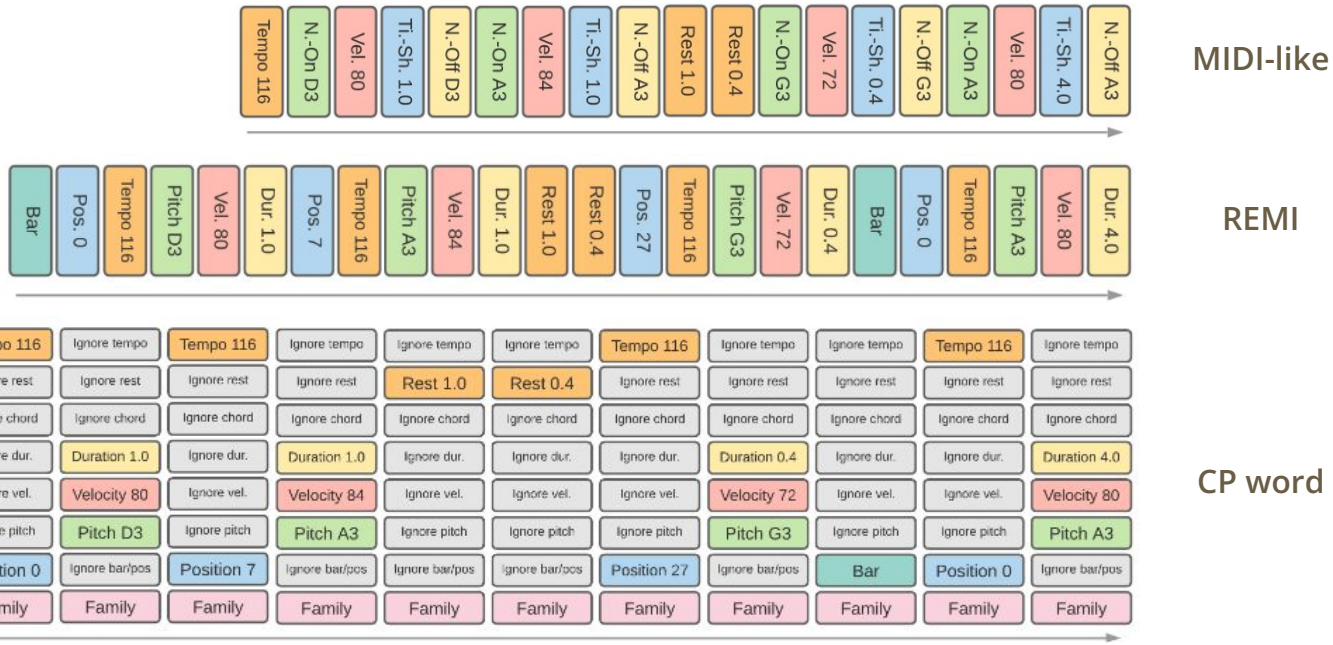
- To make the MIDI files “understandable” by the deep learning models, we convert MIDI messages into “tokens”



Y.-S. Huang and Y.-H. Yang, “Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions,” in Proc. of the ACM International Conference on Multimedia (ACMMM), 2020, pp. 1180–1188.

Token Representation

- There are lots of ways to represent one music performance
  - MIDI-like
  - REMI
  - CP word
  - etc...



Basic Task a. - Unconditional Generation

- You need to:
  - Train a Transformer-based model that can generate **32 bars** symbolic music
  - Compare model performance between different inference configurations (see the next several pages for details)

Basic Task a. - Requirements

- In your report, you should write:
  - Your token representation & model architecture
  - Implementation details (e.g. data augmentation, hyper-parameters, etc.)
  - Implement **at least 3 combinations** of your inference configurations (e.g. model loss, sampling method, temperature, etc.)
    - NOTE: you can also do ablation studies on different token representation and model architecture, but this is more time-consuming, as you need to train multiple models

Basic Task a. - Requirements

- In your report, you should write:
  - For **each** combination, generate **20** mid/midi files (**32 bars**) to calculate their average objective metrics results (see the next page)

- Submission files:
  - Choose **one** combination, generate **20** mid/midi files (**32 bars**) and convert them to **.wav** files as listening samples

Basic Task a. - Objective Metrics

- **Pitch-Class Histogram Entropy (H)**: measures erraticity of pitch usage in shorter timescales (e.g., 1 or 4 bars). (**H4 is required**)
- **Grooving Pattern Similarity (GS)**: measures consistency of rhythm across the entire piece. (**required**)
- **Structureness Indicator (SI)**: detects presence of repeated structures within a specified range of timescale. (optional)

❖ Use MusDr (TA will provide the sample code: *eval_basic.py*)

❖ The **closer** the score of generation results compare to the original dataset, the better

Basic Task a. - Objective Metrics

- Example table for objective result comparisons

Model	Representation	w/ Chord	Loss	Top-K	Temperature	H1	**H4**	**GS**	SI (short)	SI (mid)	SI (long)
GPT2	REMI	O									
Transformer-XL	CP word	X									
Real Data											

Basic Task b. - Prompt Continuation

- You need to:
  - Use the model checkpoint from Basic Task a. (no need to retrain), generate continuation of **24 bars** for **3** midi prompts provided by TA (each prompt has **8 bars**)
  - Think of this as a “Mandatory Song (指定曲)” competition

Basic Task b. - Requirements

- In your report, you should write:
  - For each prompt, use **3 different inference configurations** to generate the continuation, resulting in **8+24 bars** pieces and store them into **.wav** files as listening samples
  - Specify the model, representation and inference configurations you use for each continuation
  - No objective metrics are needed here

Basic Task b. - Requirements

- Submission files:
  - Submit all the generated .mid+.wav files, and for **each** prompt, specify the **one** you think that generates the best among different inference configurations

Basic Task - Tips

- For token representation:
  - You can use the MidiTok library as the starting point. It directly provides you some famous token representations
  - TA will also provide a simple example (*tokenization.py*) for performing tokenization from scratch. You can write your own token representation based on it if you would like to

Basic Task - Tips

- For model architecture:
  - For highest simplicity, use the Hugging Face transformers library. You can directly use the decoder-only model classes such as **GPT2LMHeadModel**.
  - Another option is to utilize open source code if you want to learn the whole pipeline of the symbolic music generation. Below are some examples:
    - 1-stage generation: Pop Music Transformer, Compound Word Transformer
    - 2-stage generation: Compose & Embellish

Basic Task - Warnings

- Start this homework as soon as possible!
  - It may take several days to train the model for it to generate beautiful music
  - For SI score, it is time-consuming to compute scape plot. Start early if you want to report it
- Use the whole POP909 training set and **don’t use other datasets**

Bonus Task - Accompaniment Generation

- You need to:
  - Use the **10** melody line (MELODY track) prompts provided by TA (each prompt has **16 bars**), generate their corresponding accompaniment (PIANO track)

Bonus Task - Tips

- For model architecture:
  - Option A: **Interleaving** MELODY & PIANO, and use the **decoder-only** model
    - Pros: training is mostly identical to the Basic Tasks
    - Cons: you should write a sophisticated inference pipeline for inserting the conditions, and the exposure of future conditions is limited (depending how you interleave the two tracks)

Bonus Task - Tips

- For the model architecture:
  - Option B: **seq2seq** approach by utilizing the **encoder-decoder** model
    - Pros: the model can see the whole conditions before generating accompaniment
    - Cons: you need to implement an encoder-decoder model
      - Hugging Face transformers library also has several encoder-decoder model classes you can use (e.g. **T5ForConditionalGeneration**)

Bonus Task - Requirements

- In your report, you should write:
  - For each prompt, generate its corresponding accompaniment using your best configuration, resulting in **16 bars MELODY+PIANO** pieces and store them into **.wav** files as listening samples
  - Specify the model, representation and inference configuration you use
  - Run objective metrics on those generated samples (see the next second page for details)
  - Do some ablation studies if you have time

Bonus Task - Requirements

- Submission files:
  - Submit all the generated .mid+.wav files

Bonus Task - Objective Metrics

- **Chord Accuracy (CA)**: measure whether the chords of generated tracks match the conditional chord sequence
- **Overlapped Area (OA)**: measure the difference between generated musical piece and ground-truth musical piece
    - **Pitch (OA_P)**, **Velocity (OA_V)**, **Duration (OA_D)** and **Inter-Onset Interval (OA_IOI)**

❖ TA will provide the sample code: *eval_bonus.py*

Yi Ren, Jinzheng He, Xu Tan, Tao Qin, Zhou Zhao, and Tie-Yan Liu. Popmag: Pop music accompaniment generation. In Proceedings of the 28th ACM International Conference on Multimedia, MM '20, page 1198–1206, New York, NY, USA, 2020. Association for Computing Machinery

Submission

- Report (to NTU COOL)
- Code, README and model checkpoints for inference (to cloud drive)
- Generation Results (to cloud drive)
  - Basic Task a.: 20 midi files + 20 wav files for the best inference configuration
  - Basic Task b.: (3 midi files + 3 wav) * 3 files for each prompt song (specify the best one for each prompt song!)
  - Bonus Task: 10 midi files + 10 wav files for each prompt song
- Don’t upload: training data, preprocessed data, others’ model, cache file

Submission - Report

- Write with PPT or PPT-like format (16:9)
- Upload studentID_report**.pdf** (ex: r12345678_report.pdf)
- Please create a report that is clear and can be understood without the need for oral explanations
- **There is no specific length requirement**, but it should clearly communicate the experiments conducted and their results. Approximately 10 pages is a suggested standard, but not a strict limitation

Submission - Code

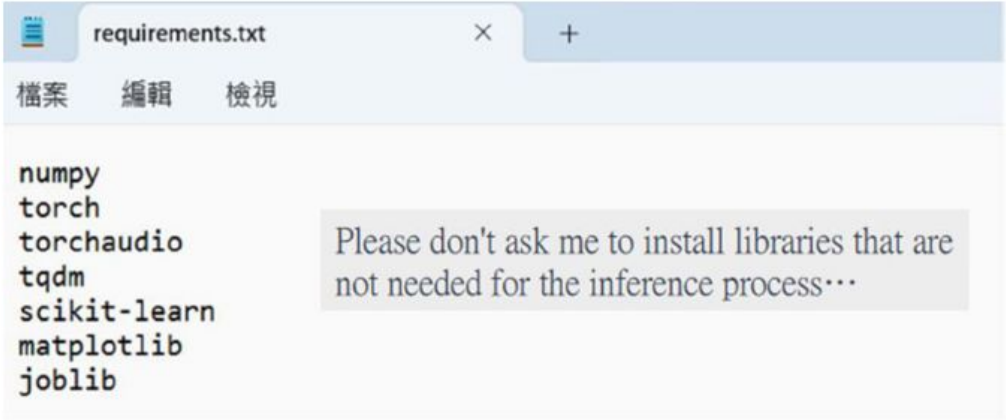
- Upload all your source code and model to a cloud drive, open access permissions, and then upload the link to the NTU Cool assignment in comments, as well as include it on the first page of the report

Submission - Code

- You will need to upload **requirements.txt**. TA will run:

```
pip install -r requirements.txt
```

If you have used third-party programs that cannot be installed directly via 'pip install,' please write the URL and install method command by command on **your readme file**.



Submission - Code

- You will also need to upload **README** file to guide me on how to perform inference on your model. TA will inference the generation from scratch, which is to make sure your generation results is not from others
- The inference code should generate some listening samples
- The inference process should allow me to set the output file path

Scoring

- HW2 accounts for 20% of the total grade
- Report: 100% (You'll get an A- if you only achieve the minimum requirements. Adding another experiment results, analysis, discussion will help you to get higher score)

When You Encounter Problems

- Check out all course materials and announcement documents
- Use the power of the internet and AI
- Use Discussions on NTU COOL
- Email me (*rakechen168@gmail.com*) or come to the office hour

Rules

- Don’t cheat
- Use Transformer-based generation model
- Train the model from scratch
- Can use public codes with citation in report
- Don’t use extra data

Timeline

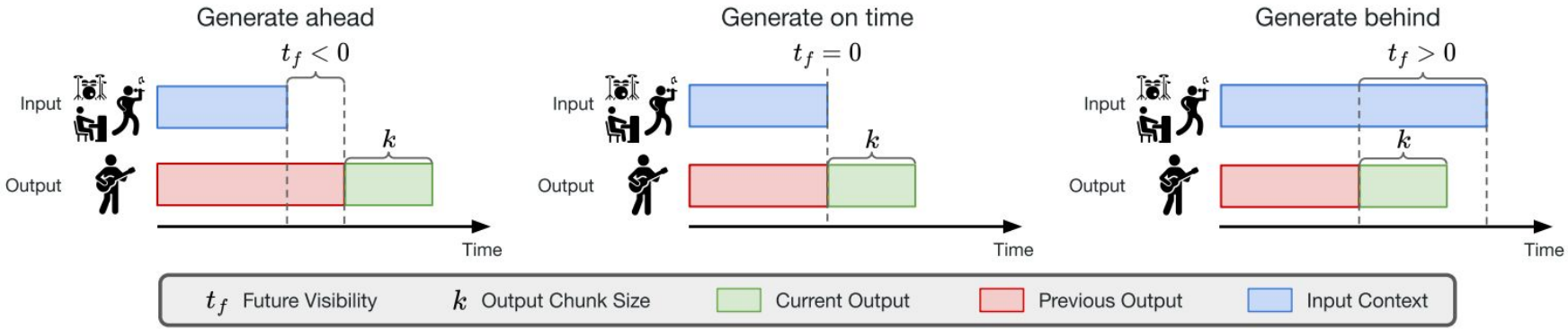
- Week 5 (10/6 Tues.) - Announcement of HW2
- Week 8 (10/26 Mon. 23:59) - Deadline

Late submission: 1 day (-20%), 2 days (-40%), 3 days (-60%), after that (no score)

All You Need to Do Before Deadline

- Report
  - StudentID_report.pdf
- Cloud drive link
  - generation results
  - README file
  - requirements.txt
  - Codes and model to run inference
  - Others codes

Appendix: Toward Streaming Accompaniment Generation



Y. Wu, M. Wang, H. Lei, S. Brade, L. Blanchard, S.-L. Wu, A. C. Courville, and C.-Z. A. Huang, “Streaming generation for music accompaniment,” arXiv preprint arXiv:2510.22105, 2025.